```
U, S, V = sp.linalg.svd(A)
print(U@np.diag(S)@V)
```

```
[[ 11.  22.  33.]
 [ 44.  55.  66.]
 [ 77.  88. 888.]]
```

Figure 7: Singular value decomposition with SciPy

discuss it here for the time being.

A few more Python libraries for AI and machine learning

We now discuss two more libraries for developing AI and machine learning based applications. There is a reason why I didn't say Python libraries. But we will come to that later. If you speak to a layperson about AI, what do you think will be the first image that will come to his/her mind? It will probably be a Terminator like scenario, where a machine can identify a person just by looking at him/her. Computer vision is one of the most important domains where AI and machine learning based applications are deployed a lot. So we are now going to get familiar with two libraries used in computer vision — OpenCV and Matplotlib. OpenCV (open source computer vision) is a library used mainly for real-time computer vision, and is developed using C and C++. C++ is the primary interface of OpenCV and that is the reason why I didn't call it a Python library. However, Matplotlib is a plotting library for Python. One of my earlier articles in OSFY covers Matplotlib in a more detailed manner *(https://www.opensourceforu.com/2018/05/scientific-graphics-visualisation-an-introduction-to-matplotlib)*.

There is an important reason for introducing these two libraries now. I have been preaching all this while that matrices are very important, but now I am going to give you a practical example of that. To illustrate this, let us read an image from our computer to a Jupyter Notebook, for further processing. We use Matplotlib for this task. Install Matplotlib with the command *pip install matplotlib*, if it is not available in your Jupyter Notebook. Figure 8 shows the code and the output that reads an image.

```
[10]: from matplotlib import pyplot as plt
      import matplotlib.image as mimg
      image = mimg.imread("OSFY-Logo.jpg")
      plt.imshow(image)
      plt.show()
```


```
[23]: print(type(image))
      print(image.shape)
```

```
<class 'numpy.ndarray'>
(80, 270, 3)
```

Figure 8: Reading and displaying an image with Matplotlib

In the figure, lines 1 and 2 import some functions from Matplotlib. Notice that, if required, you can import individual functions or packages from a library instead of importing the whole library. These two lines contain basic Python code. Line 3 reads an image titled *'OSFY-Logo.jpg'* from my computer. I have downloaded this image from the home page of the OSFY portal. This image is of height 80 pixels and width 270 pixels. Lines 4 and 5 display the image on a Jupyter Notebook window. But what is more important are the two lines of code shown below the image (marked with a red rectangle). The output of these lines of code tells us that the variable named image is actually a NumPy array. Further, it is a three-dimensional array of shape 80 x 270 x 3.

The two-dimensional array (80 x 270) part is clear from the size of the image mentioned earlier. But what about the third dimension? This is because of the fact that the image we saw is a colour image. Colour images in computers are often stored using the RGB colour model, where one layer is used for each of the three primary colours — red, green, and blue. I am sure all of you remember the experiments during our school days where primary colours were mixed to form different colours. For example, red and green when combined give yellow. The varying shades of each colour are often represented with numbers ranging from 0 (darkest) to 255 (brightest). Thus, a pixel with value (255, 255, 255) represents pure white colour.

Now, execute the command *print(image)*. Parts of a large array will be shown in the Jupyter Notebook and you will see a lot of 255s printed in the beginning of the array. What is the reason for this? If you look at the logo of *OSFY*, you will understand the reason. There is a lot of white colour in the borders of the logo and hence a lot of 255s are printed in the beginning. On a side note, it is really informative to learn more about RGB colour models as well as other colour models like CMY, CMYK, HSV, etc.

We now do the reverse process. We will create an image from an array. First, consider the code shown in Figure 9. It shows how two 3 x 3 matrices filled with random values between 0 and 255 are generated. Notice that though the same code is executed twice, the results are different. A pseudo-random number generator function of NumPy called *randint* does this. Indeed, the chances of my winning a lottery are far more than that of the two matrices being exactly equal.

Our next plan is to generate a three-dimensional array of shape 512 x 512 x 3 and then convert it into an image. But for that we are going to use the library OpenCV. Be careful while installing OpenCV. The command for installing it is *pip install opencv-python*. Now